

Transfer Learning with PyTorch

VGG16, Fashion-MNIST Preprocessing, Frozen Features, and Reliable Evaluation

Abstract

Transfer learning reuses a representation learned from a large source task to solve a related target task with less target data and computation. These notes construct a VGG16-based Fashion-MNIST classifier from first principles. The complete tensor path converts a flat 784-pixel row into a normalized image tensor of shape $[3, 224, 224]$, batches it as $[B, 3, 224, 224]$, applies frozen ImageNet feature layers, flattens $512 \times 7 \times 7 = 25,088$ features, and trains a new $25,088 \rightarrow 1,024 \rightarrow 512 \rightarrow 10$ classifier. Every shape, data type, normalization constant, gradient boundary, optimizer responsibility, training state, and evaluation step is defined and checked. The project context includes a 93.1% optimized CNN recap and an earlier approximate 94% ten-epoch transfer result. A separate live training attempt ended after six epochs when hosted compute was exhausted, so no final metric is invented for that interrupted run. Modern PyTorch APIs, executed implementation checks, failure modes, validation discipline, and extension paths through ResNet, MobileNet, and Inception complete the treatment.

Contents

1	Purpose and complete mental map	2
1.1	The practical problem	2
1.2	Why transfer learning is useful	3
1.3	Project evidence and a precise goal	3
2	What is transferred and why it can work	4
2.1	A mathematical decomposition	4
2.2	Feature hierarchy	4
2.3	Relatedness and negative transfer	4
3	VGG16 and the target tensor contract	5
3.1	Architecture responsibility	5
3.2	Why the original classifier cannot remain	6
3.3	Expanded parameter count	6
4	The five-step implementation workflow	7
5	Preprocessing Fashion-MNIST for VGG16	8
5.1	Why the original table is incompatible	8
5.2	Transforms as one ordered function	9
5.3	Normalization from formula to numbers	9
6	Dataset and DataLoader responsibilities	10
6.1	Object state and invariants	10

6.2	Custom dataset implementation	11
6.3	Load, split, and batch the table	12
7	Load, freeze, and replace VGG16	13
7.1	Version-safe model construction	13
7.2	Freezing does not remove forward computation	13
7.3	Executed contract checks	14
8	Loss, training, and evaluation	14
8.1	Cross-entropy from logits	14
8.2	Training algorithm	15
8.3	Evaluation state and accuracy	16
9	Executed verification and failure modes	17
9.1	What the bounded implementation check established	17
9.2	Common failures and their causes	17
10	Interpreting the available results honestly	18
10.1	A defensible comparison protocol	18
10.2	Other pretrained backbones	19
11	Optional advanced extensions	19
11.1	Gradual unfreezing	19
11.2	Feature caching	19
11.3	Data augmentation	20
12	Consolidated takeaways and review questions	20

1 Purpose and complete mental map

1.1 The practical problem

Fashion-MNIST is a ten-class image-classification dataset of clothing and footwear. One grayscale image contains $28 \times 28 = 784$ pixel values. A previous convolutional neural network (CNN) already improved the project substantially, but further gains by training a larger network from random initialization would require more experiments, more compute, and possibly more labeled examples.

A **convolutional neural network** is a model that learns local spatial filters on image grids. A **class** is one possible output category. A **parameter** is a learned numerical value such as a filter weight or bias. A **pretrained model** is a model whose parameters were already fitted on an earlier dataset.

Definition

Transfer learning is the reuse of knowledge learned for a source task in a different but related target task. The **source task** supplies pretrained parameters; the **target task** supplies the new prediction goal and target data.

The source model is VGG16 trained on ImageNet object classification. The target problem is Fashion-MNIST classification. The source output has 1,000 ImageNet classes, whereas the target output has ten

Fashion-MNIST classes. The source classifier must therefore be replaced.

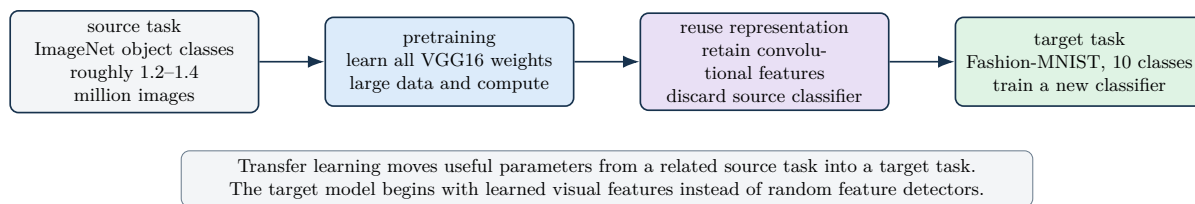


Figure 1: A large source task supplies learned visual features; the target task reuses those features and learns a smaller task-specific classifier. [Technical reference](#).

1.2 Why transfer learning is useful

Deep neural networks have two common practical costs:

Data cost. A model with many parameters usually needs many representative labeled examples to generalize reliably.

Compute cost. Repeated forward passes, backward passes, and parameter updates can require expensive accelerators and long training runs.

Pretraining pays much of this cost once on a large source dataset. A target project can then reuse the learned representation and fit only a smaller classifier. This does not remove all costs: every target image must still pass through the feature extractor, and related source and target tasks are important.

Intuition

Learning to balance and monitor the road on a bicycle can make learning to ride a motorcycle easier. The two tasks are not identical, but they share useful prior skills. A person without the related skill must learn more from the beginning. Transfer learning follows the same structure: reuse shared competence and learn only what the target task changes.

1.3 Project evidence and a precise goal

Configuration	Reported result	Correct interpretation
Basic CNN	about 92.6%	image structure improved on the flattened ANN sequence
Optimized CNN	93.1%	augmentation, dropout, and hyperparameter search improved the baseline
Earlier VGG16 transfer run	about 94% after 10 epochs	approximate prior evidence, not the interrupted run below
Live VGG16 attempt	stopped after epoch 6	hosted compute ended before final evaluation; no final accuracy exists

The goal is to understand and implement the method, not merely to quote the largest number. A trustworthy comparison also requires validation data for model decisions and an untouched test set for one final report.

The suggestion that additional hyperparameter trials might approach 93.5% was a possible future outcome, not a completed measurement. It must remain separate from the observed 93.1% recap value.

Takeaway

Transfer learning starts from reusable visual features, aligns the input and output contracts with the target task, trains only the intended parameters, and reports only measurements that actually completed.

2 What is transferred and why it can work

2.1 A mathematical decomposition

Write the model as two functions:

$$h = f_{\theta_f}(x), \quad z = g_{\theta_h}(h).$$

Here x is one input image; f is the feature extractor; θ_f is the collection of feature-extractor parameters; h is the resulting feature vector; g is the classifier head; θ_h is the collection of head parameters; and z is the vector of class logits. A **logit** is an unrestricted real-valued class score.

The displayed equations compute the model in two stages. First, f_{θ_f} converts pixels into features. Second, g_{θ_h} converts features into target-class scores. Their importance is that the two parameter groups can be trained differently.

For fixed-feature transfer, the source parameters remain constant and only the head is optimized:

$$\theta_h^* = \arg \min_{\theta_h} \frac{1}{N} \sum_{i=1}^N \ell(g_{\theta_h}(f_{\theta_f^{\text{pre}}}(x_i)), y_i).$$

In this expression, N is the number of target training examples; i indexes one example; x_i is its image; y_i is its correct class index; θ_f^{pre} denotes fixed pretrained feature parameters; ℓ is the per-example loss; and θ_h^* is the head parameter set that minimizes average target loss. The equation matters because gradients need not update θ_f^{pre} .

Technical precision

Training only a replacement classifier is most precisely called **fixed-feature extraction**. The term **fine-tuning** is often used broadly, but in a narrower technical sense fine-tuning means unfreezing at least some pretrained layers and updating them on the target data. Both are forms of transfer learning.

2.2 Feature hierarchy

Early CNN filters often respond to low-level patterns such as edges, orientations, and color contrasts. Intermediate layers combine them into textures, corners, or parts. Deeper layers can represent complex source-task patterns. Because most natural images share edges and textures, early and intermediate features can remain useful on a new image task.

Freezing also reduces overfitting risk when the target dataset is small. The target optimizer cannot freely distort millions of feature parameters using limited evidence. However, freezing can be too restrictive when the target images differ strongly from ImageNet.

2.3 Relatedness and negative transfer

Transfer works best when source and target representations are compatible. Everyday photographs and clothing images both contain edges, textures, and shapes, so ImageNet features are a plausible starting point. **Negative transfer** occurs when reused knowledge harms target performance. Examples include

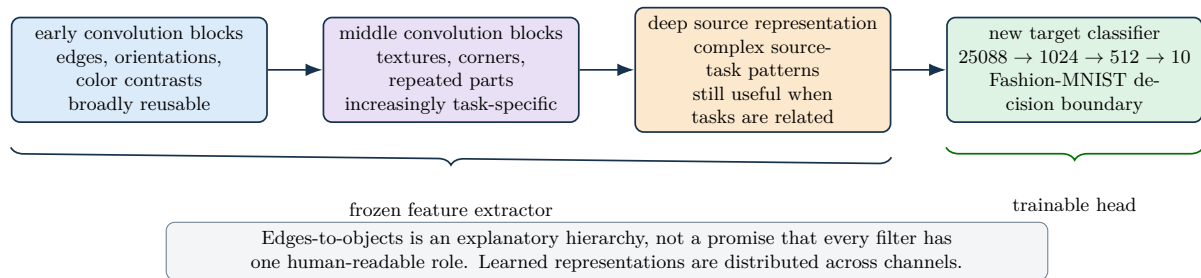


Figure 2: The target classifier is replaced while the convolutional representation is frozen; the common edges-to-objects description is useful intuition rather than a one-filter-one-concept guarantee. [Technical reference](#).

source images with very different physical structure, incompatible channels, or preprocessing that changes the meaning of pixel values.

A practical strategy depends on target data size and similarity:

Target situation	Starting strategy	Reason
Small and similar	freeze the backbone; train a head	limit variance and computation
Moderate and similar	train a head, then unfreeze the last block	adapt high-level features cautiously
Large and similar	fine-tune more blocks with a small learning rate	target data can support more parameters
Very different	compare transfer with training a suitable model from scratch	source features may be mismatched

3 VGG16 and the target tensor contract

3.1 Architecture responsibility

VGG16 is a CNN architecture with five convolutional blocks followed by a classifier. The number 16 refers to its weight-bearing convolutional and fully connected layers in the original design. In TorchVision the model exposes three important components:

features. Convolution, ReLU, and max-pooling operations that extract spatial features.

avgpool. Adaptive pooling that produces a fixed 7×7 spatial grid.

classifier. Fully connected layers that produce source-class logits.

The published VGG16 ImageNet weight specification reports top-1 accuracy 71.592%, top-5 accuracy 90.382%, 138,357,544 parameters in the original model, approximately 15.47 billion floating-point operations per forward pass, and a 527.8 MB weight file. **Top-1 accuracy** requires the highest-scoring class to be correct; **top-5 accuracy** requires the correct class to appear among the five highest scores. These are ImageNet source metrics, not Fashion-MNIST target results.

The implementation specification gives a minimum spatial size of 32×32 because adaptive pooling can accept more than one input size. The selected pretrained inference recipe nevertheless resizes to 256 and center-crops to 224×224 so target inputs follow the statistics under which the weights are documented.

The target batch must enter in channel–height–width order:

$$X \in \mathbb{R}^{B \times 3 \times 224 \times 224}.$$

Here X is the image batch; B is mini-batch size; 3 is the red–green–blue channel count; and 224 is both height and width. The feature extractor returns

$$F \in \mathbb{R}^{B \times 512 \times 7 \times 7},$$

where F contains 512 feature maps for every example. Flattening preserves B and multiplies the remaining dimensions:

$$512 \times 7 \times 7 = 25,088, \quad H \in \mathbb{R}^{B \times 25,088}.$$

The tensor H is the flattened feature matrix. Every row contains 25,088 features for one image. This value determines the first replacement linear layer's input width.

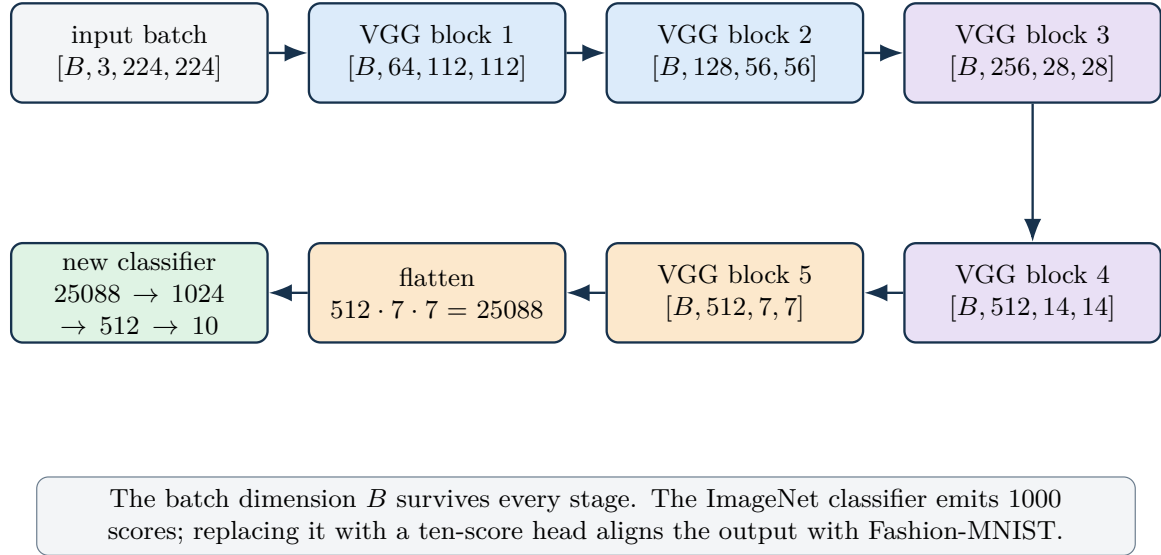


Figure 3: VGG16 converts a three-channel 224×224 batch into 25,088 flattened features before the ten-class replacement head. [Technical reference](#).

3.2 Why the original classifier cannot remain

The original ImageNet head returns

$$Z_{\text{source}} \in \mathbb{R}^{B \times 1000},$$

whereas Fashion-MNIST requires

$$Z_{\text{target}} \in \mathbb{R}^{B \times 10}.$$

The final dimension counts class scores. A 1,000-score output does not match ten target labels, even if the feature extractor is useful. The replacement head is

$$25,088 \longrightarrow 1,024 \longrightarrow 512 \longrightarrow 10.$$

Each arrow denotes a fully connected linear transformation. ReLU activation and dropout follow the first two linear layers; the final layer emits logits without softmax.

3.3 Expanded parameter count

A linear layer with d_{in} inputs and d_{out} outputs owns

$$P = d_{\text{in}}d_{\text{out}} + d_{\text{out}}$$

parameters. The product counts weights and the final term counts one bias per output. Applying the formula to the replacement head gives

$$\begin{aligned} P_1 &= 25,088 \cdot 1,024 + 1,024 = 25,691,136, \\ P_2 &= 1,024 \cdot 512 + 512 = 524,800, \\ P_3 &= 512 \cdot 10 + 10 = 5,130. \end{aligned}$$

The symbols P_1 , P_2 , and P_3 count parameters in the three target linear layers. Their sum is

$$P_{\text{head}} = 25,691,136 + 524,800 + 5,130 = 26,221,066.$$

This calculation matters because “only training the head” still trains more than 26 million parameters. The head is smaller than the original VGG16 classifier, but it is not a tiny model.

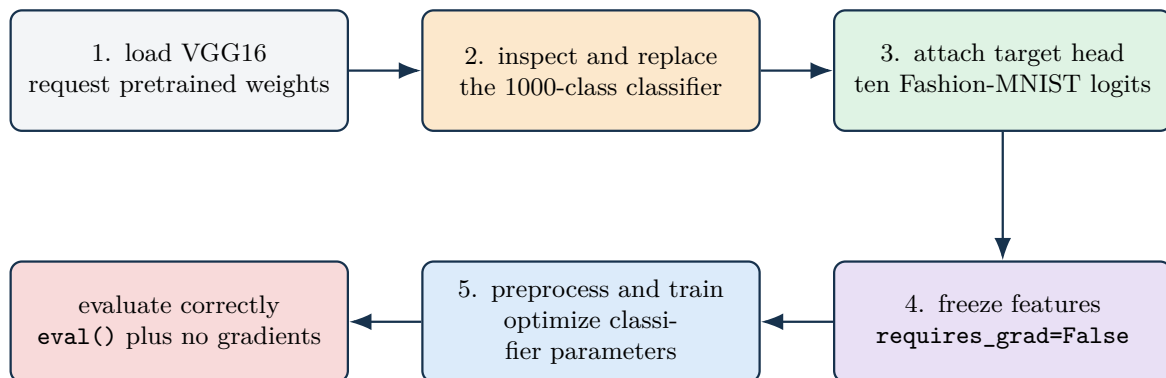
Shape and state check

The first replacement layer must receive exactly $512 \cdot 7 \cdot 7 = 25,088$ features. Changing crop size or bypassing VGG16’s adaptive pool can change the feature width and cause a matrix-multiplication error.

4 The five-step implementation workflow

A reliable implementation separates model construction from data preparation and training:

1. load VGG16 with pretrained ImageNet weights;
2. inspect and replace its 1,000-class classifier;
3. attach a ten-class target head;
4. freeze the feature parameters;
5. preprocess target images and optimize only the target head.



A model is not ready merely because it loads. Input statistics, channel count, spatial size, output width, gradient flags, optimizer scope, and evaluation state must all agree.

Figure 4: Loading, replacing, freezing, preprocessing, training, and evaluation form one connected contract; omitting any contract check can invalidate the run. [Technical reference](#).

The older interface `models.vgg16(pretrained=True)` requests pretrained weights but is deprecated in current TorchVision. The explicit modern interface is `vgg16(weights=VGG16_Weights.DEFAULT)`. Explicit weight enums identify both the parameters and their associated preprocessing recipe.

5 Preprocessing Fashion-MNIST for VGG16

5.1 Why the original table is incompatible

One CSV row stores a label and 784 grayscale pixel columns. VGG16 expects a three-channel image batch with ImageNet-compatible spatial size and normalization. The complete conversion for one image is

$$[784] \rightarrow [28, 28] \rightarrow [28, 28, 3]_{\text{HWC}} \rightarrow [3, 224, 224]_{\text{CHW}}.$$

HWC means height–width–channel, the common array order accepted by the Python Imaging Library (PIL). CHW means channel–height–width, the order used for one PyTorch image tensor. The conversion also changes data type from eight-bit integers to normalized 32-bit floating-point values.

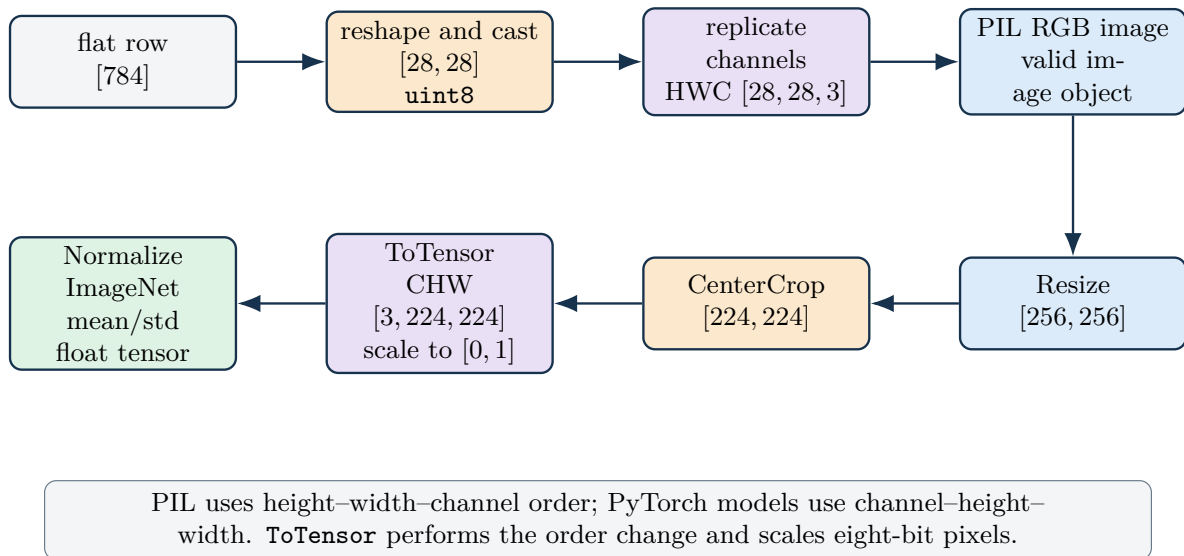


Figure 5: The preprocessing path reconstructs the image, creates three channels, converts HWC to CHW, resizes and crops, scales byte values, and applies ImageNet normalization. [Technical reference](#).

The explicit steps are:

1. reshape 784 values into 28×28 ;
2. cast to `numpy.uint8` for a conventional eight-bit image path;
3. replicate the grayscale grid three times;
4. place channels last so the array shape is $[28, 28, 3]$;
5. create a PIL RGB image;
6. resize to 256×256 and take the central 224×224 crop;
7. apply **ToTensor**, producing float CHW data and scaling bytes to $[0, 1]$;
8. normalize each channel using ImageNet statistics.

Technical precision

PIL supports several image modes and data types; `uint8` is not its only possible input. It is the appropriate, predictable representation for this RGB byte-image path. Likewise, **ToTensor** scales `uint8` image data, but arbitrary floating arrays require explicit range discipline.

5.2 Transforms as one ordered function

```
from torchvision import transforms

IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

custom_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=IMAGENET_MEAN,
        std=IMAGENET_STD,
    ),
])
```

Listing 1: Define the ImageNet-compatible target transform

`Compose` applies its functions in list order. `Resize` changes spatial dimensions, `CenterCrop` selects a central region, `ToTensor` converts format and range, and `Normalize` changes channel statistics. Defining a transform does not apply it; application occurs when the dataset retrieves a sample.

5.3 Normalization from formula to numbers

For channel c , normalization computes

$$z_c = \frac{p_c - \mu_c}{\sigma_c}.$$

Here p_c is a scaled pixel in $[0, 1]$; μ_c is the ImageNet mean for channel c ; σ_c is the corresponding ImageNet standard deviation; and z_c is the normalized model input. Subtraction centers the value and division expresses its distance from the source mean in source-standard-deviation units.

For a replicated byte value 128,

$$p = \frac{128}{255} = 0.5019607843.$$

The three normalized values are

$$\begin{aligned} z_R &= \frac{0.5019607843 - 0.485}{0.229} \approx 0.0741, \\ z_G &= \frac{0.5019607843 - 0.456}{0.224} \approx 0.2052, \\ z_B &= \frac{0.5019607843 - 0.406}{0.225} \approx 0.4265. \end{aligned}$$

The subscripts R , G , and B denote red, green, and blue. The raw channels are identical copies, but channel-specific source statistics make the normalized values different.

Pitfall

Dividing the whole CSV by 255 before `ToTensor` and then using an image conversion path designed for bytes can scale values incorrectly or change PIL mode. In this pipeline the original byte-like values remain intact until `ToTensor` performs the scaling.

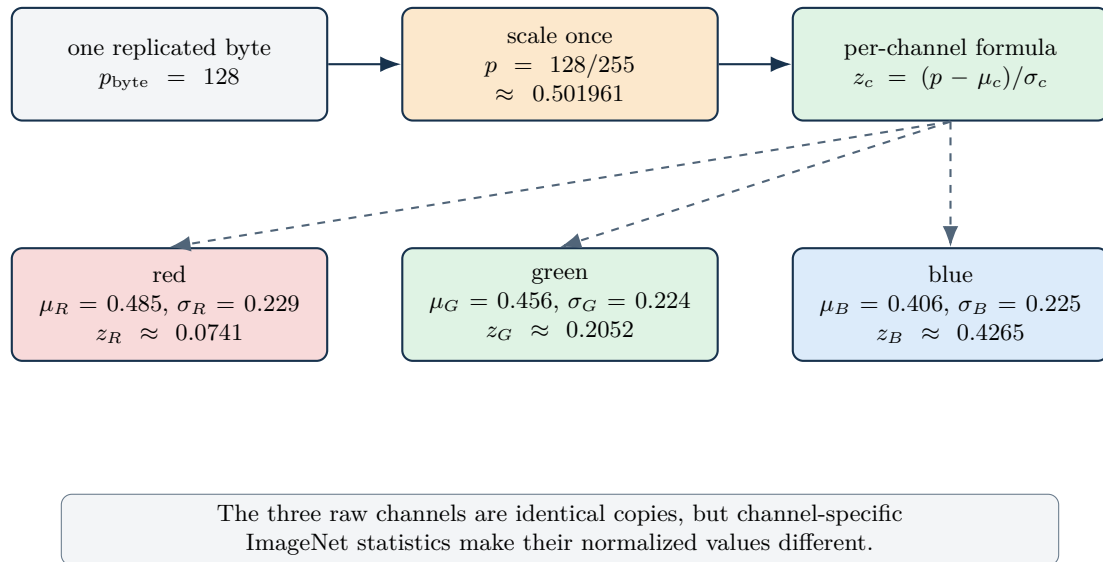


Figure 6: A byte value is first scaled once and then standardized separately with red, green, and blue ImageNet statistics. [Technical reference](#).

6 Dataset and DataLoader responsibilities

6.1 Object state and invariants

A PyTorch **Dataset** answers two questions: how many examples exist, and what one indexed example contains. Its state owns aligned feature rows, aligned labels, and the transformation function. The essential invariant is

$$\text{len}(\text{features}) = \text{len}(\text{labels}) = N,$$

where N is the dataset row count. If this equality fails, features and labels cannot be paired reliably.

`__getitem__(i)` returns one image of shape $[3, 224, 224]$ and one scalar class index. A **DataLoader** stacks B such samples into images $[B, 3, 224, 224]$ and labels $[B]$.

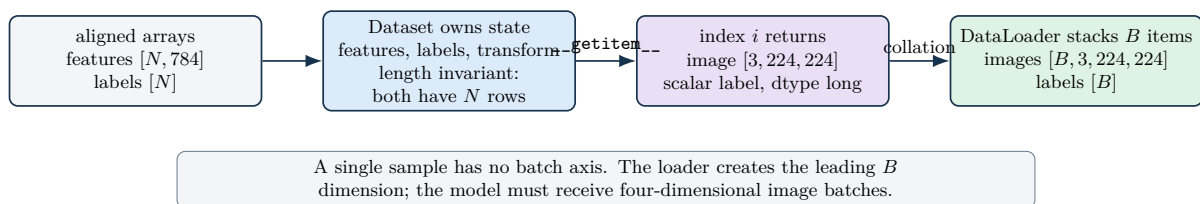


Figure 7: The dataset converts one indexed table row; the loader adds the leading mini-batch dimension and preserves label alignment. [Technical reference](#).

6.2 Custom dataset implementation

```
import numpy as np
import torch
from PIL import Image
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, features, labels, transform):
        if len(features) != len(labels):
            raise ValueError("features and labels must align")
        self.features = features
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        image = self.features[index].reshape(28, 28)
        image = image.astype(np.uint8)
        image = np.stack([image] * 3, axis=-1)
        image = Image.fromarray(image)
        image = self.transform(image)

        label = torch.tensor(
            self.labels[index], dtype=torch.long
        )
        return image, label
```

Listing 2: Convert a flat Fashion-MNIST row into one VGG-compatible sample

The `axis=-1` argument places the three copies last, producing HWC rather than CHW. PIL consumes HWC. ToTensor later returns CHW. The label uses `torch.long`, which is PyTorch’s 64-bit integer type and the required class-index type for cross-entropy.

6.3 Load, split, and batch the table

```
import pandas as pd
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader

df = pd.read_csv("fashion-mnist_train.csv")
X = df.iloc[:, 1:].to_numpy()
y = df.iloc[:, 0].to_numpy()

X_train, X_holdout, y_train, y_holdout = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

train_dataset = CustomDataset(
    X_train, y_train, custom_transform
)
holdout_dataset = CustomDataset(
    X_holdout, y_holdout, custom_transform
)

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    pin_memory=True,
)
holdout_loader = DataLoader(
    holdout_dataset,
    batch_size=32,
    shuffle=False,
    pin_memory=True,
)
```

Listing 3: Load the CSV, preserve byte pixels, split rows, and build loaders

With 60,000 rows and a 20% holdout fraction,

$$60,000 \times 0.8 = 48,000, \quad 60,000 \times 0.2 = 12,000.$$

The first product counts training rows and the second counts holdout rows. `shuffle=True` changes training order each epoch. Evaluation order need not be shuffled. Pinned memory can accelerate host-to-CUDA copies, but it does not move data to a graphics processing unit (GPU) by itself.

Pitfall

If architecture, learning rate, crop policy, or stopping epoch is selected using the holdout score, that partition has become validation data. A separate untouched test set is then required for final reporting.

7 Load, freeze, and replace VGG16

7.1 Version-safe model construction

```
import torch.nn as nn
from torchvision.models import VGG16_Weights, vgg16

weights = VGG16_Weights.DEFAULT
model = vgg16(weights=weights)

for parameter in model.features.parameters():
    parameter.requires_grad = False

model.classifier = nn.Sequential(
    nn.Linear(512 * 7 * 7, 1024),
    nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(1024, 512),
    nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(512, 10),
)
```

Listing 4: Load VGG16, freeze its features, and install the ten-class head

`requires_grad=False` prevents autograd from storing gradients for frozen feature parameters. Replacing `model.classifier` registers the new modules and their trainable parameters inside the model. Dropout with probability 0.5 randomly suppresses half of its input activations in expectation during training; it is disabled in evaluation state.

The weight enum also provides an official transform through `weights.transforms()`. The explicit transform used above is educational because every operation is visible. In production, using the weight-provided transform reduces the chance of constants drifting from the selected weight version.

7.2 Freezing does not remove forward computation

Every input still passes through all convolutional blocks. Freezing changes the backward and update responsibilities, not the forward data flow.

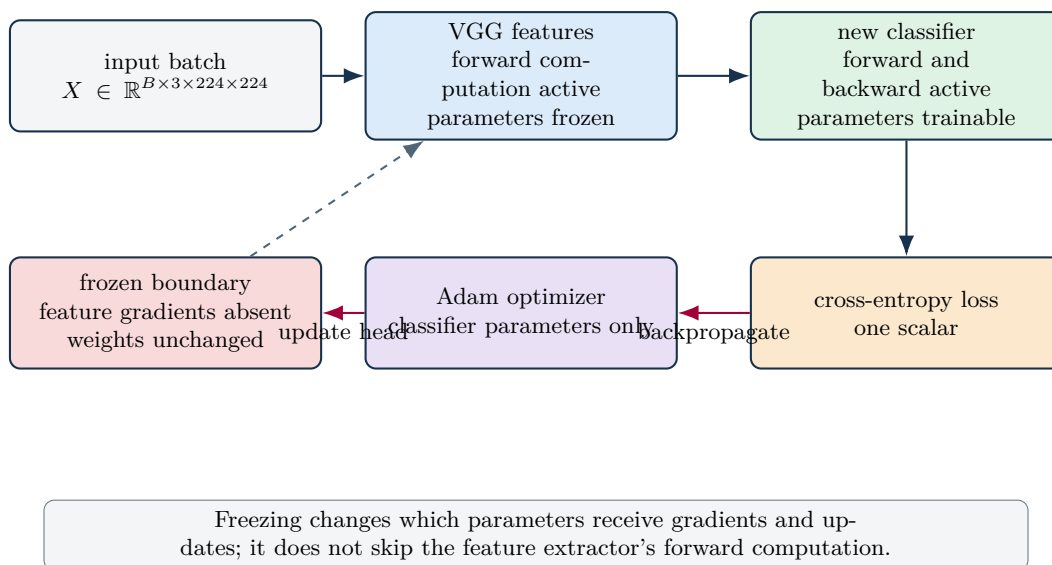


Figure 8: The frozen feature extractor remains active in the forward pass, while gradient computation and Adam updates are confined to the replacement head. [Technical reference](#).

An optimizer can be given all model parameters, but explicitly selecting the classifier makes the intended update boundary auditable:

```
import torch
import torch.optim as optim

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
model = model.to(device)

learning_rate = 0.0001
epochs = 10
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(
    model.classifier.parameters(),
    lr=learning_rate,
)
```

Listing 5: Move the model to a device and optimize the classifier only

A learning rate is the step-size control used by the optimizer. Reused features usually call for cautious learning rates, especially after any feature layers are unfrozen. The recorded head-only configuration uses 10^{-4} ; this is a starting configuration rather than a universal optimum.

7.3 Executed contract checks

```
batch_images, batch_labels = next(iter(train_loader))
assert batch_images.shape[1:] == (3, 224, 224)
assert batch_labels.dtype == torch.long

batch_images = batch_images.to(device)
with torch.inference_mode():
    features = model.features(batch_images)
    pooled = model.avgpool(features)
    flattened = torch.flatten(pooled, 1)
    logits = model.classifier(flattened)

assert features.shape[1:] == (512, 7, 7)
assert flattened.shape[1] == 25_088
assert logits.shape == (batch_images.shape[0], 10)
assert all(
    not p.requires_grad for p in model.features.parameters()
)
assert all(
    p.requires_grad for p in model.classifier.parameters()
)
```

Listing 6: Verify shape, output width, and gradient boundaries before training

These assertions fail early if channel order, crop size, flatten width, output class count, target type, or freezing state is wrong. Early contract failures are cheaper to diagnose than a long training run with an invalid setup.

8 Loss, training, and evaluation

8.1 Cross-entropy from logits

For one example with K classes and correct class y , cross-entropy is

$$\ell(z, y) = -\log \left(\frac{e^{z_y}}{\sum_{k=1}^K e^{z_k}} \right).$$

Here $z = (z_1, \dots, z_K)$ is the logit vector; z_y is the correct-class logit; k indexes all classes; e^{z_k} converts a logit into a positive exponential score; the fraction is the softmax probability assigned to the correct class; and ℓ is the scalar loss. Lower loss means more probability is assigned to the correct class.

For logits $[2, 1, 0]$ and correct class 0,

$$\sum_k e^{z_k} = e^2 + e^1 + e^0 \approx 11.10734,$$

$$p_0 = \frac{e^2}{11.10734} \approx 0.665241, \quad \ell = -\log(0.665241) \approx 0.407606.$$

The numerical example follows the same order as the compact equation. `CrossEntropyLoss` performs the stable log-softmax and negative-log operation internally; softmax must not be applied to the model output first.

8.2 Training algorithm

One mini-batch update has seven responsibilities:

1. enter training state;
2. move images and labels to the model device;
3. clear stale classifier gradients;
4. compute logits;
5. compute scalar loss;
6. backpropagate classifier gradients;
7. let Adam update classifier parameters.

```
for epoch in range(epochs):
    model.train()
    total_loss = 0.0

    for images, labels in train_loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    average_loss = total_loss / len(train_loader)
    print(
        f"Epoch {epoch + 1}/{epochs}, "
        f"loss={average_loss:.6f}"
    )
```

Listing 7: Train the replacement classifier for a bounded number of epochs

`zero_grad` is necessary because PyTorch accumulates gradients by default. `backward` applies the chain rule. `step` mutates the trainable head parameters. The average shown divides the sum of per-batch mean losses by the number of batches; for a final short batch of different size, a sample-weighted average is more exact.

8.3 Evaluation state and accuracy

Accuracy is

$$A = \frac{C}{N},$$

where C is the number of correct predictions, N is the number of evaluated examples, and A is the fraction correct. Multiplying A by 100 expresses a percentage.

```
def accuracy(model, data_loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.inference_mode():
        for images, labels in data_loader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            logits = model(images)
            predictions = logits.argmax(dim=1)
            correct += (predictions == labels).sum().item()
            total += labels.numel()

    return correct / total

holdout_accuracy = accuracy(model, holdout_loader, device)
print(f"holdout accuracy={holdout_accuracy:.6f}")
```

Listing 8: Evaluate without dropout randomness or gradient recording

`model.eval()` changes dropout behavior. `torch.inference_mode()` disables gradient recording and related overhead. These operations have different responsibilities and reliable evaluation uses both.

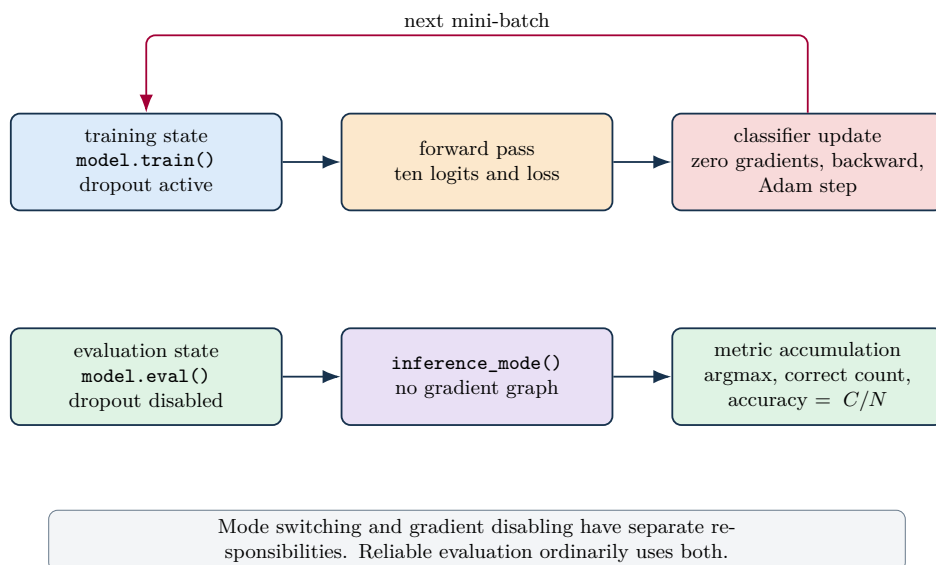


Figure 9: Training activates dropout and mutates the classifier; evaluation disables dropout, records no gradients, and accumulates predictions without updates. [Technical reference](#).

9 Executed verification and failure modes

9.1 What the bounded implementation check established

An isolated PyTorch/TorchVision run exercised the complete implementation contract without inventing pretrained accuracy. To avoid an unnecessary 528 MB weight download, the runtime instantiated the same VGG16 architecture with random weights; this verifies code, shapes, freezing, gradients, updates, and state, but not the benefit of ImageNet parameters.

The executed checks established:

- one flat Fashion-MNIST row became a float tensor $[3, 224, 224]$ and a scalar long label;
- one VGG batch followed $[1, 3, 224, 224] \rightarrow [1, 512, 7, 7] \rightarrow [1, 25,088] \rightarrow [1, 10]$;
- the frozen feature extractor contained 14,714,688 parameters;
- the trainable classifier contained 26,221,066 parameters;
- one real Adam step changed sampled classifier weights while the first frozen convolution remained unchanged and had no gradient;
- repeated training-state outputs differed because of dropout, while repeated evaluation-state outputs were identical;
- the cached canonical Fashion-MNIST training split contained 60,000 examples and one real image produced finite ten-class logits.

Shape and state check

Architecture-only execution with random weights validates implementation mechanics, not transfer-learning performance. A transfer metric requires the actual pretrained weight enum, target training, and completed evaluation.

9.2 Common failures and their causes

Failure	Violated contract	Repair
one-channel batch enters VGG16	first convolution expects 3 channels	replicate grayscale or use an adapted first layer with justified retraining
raw 28×28 image enters VGG16	repeated pooling cannot preserve the required feature geometry	resize and crop to the selected weight recipe
three-channel normalization receives one channel	mean/std length and channel count disagree	create three channels before normalization
CHW NumPy array is sent to PIL	PIL expects HWC for ordinary RGB arrays	use <code>axis=-1</code> or transpose explicitly
floating class targets enter cross-entropy	class-index targets must be long integers	cast labels to <code>torch.long</code>
targets have shape $[B, 1]$	class-index targets must have shape $[B]$	remove the singleton dimension
<code>self.label</code> is returned after storing <code>self.labels</code>	object state uses inconsistent attribute names	use one plural attribute consistently in construction and retrieval
original classifier remains	output width is 1000 rather than 10	replace and verify the final layer
all parameters enter a high-rate optimizer	pretrained features may be overwritten	freeze first; select head parameters; unfreeze deliberately

Additional silent risks include applying scaling twice, forgetting `eval()`, using a target split repeatedly as a test set, and comparing architectures with different data splits. Assertions and a validation protocol prevent these errors from hiding behind a plausible loss curve.

10 Interpreting the available results honestly

The optimized from-scratch CNN reached 93.1% in the project recap. An earlier completed VGG16 configuration was described as approximately 94% after ten epochs. These values suggest a transfer-learning gain, but the word “approximately” limits precision and the experiments should share the same split before a strict comparison is made.

A separate live training attempt stopped after epoch 6 because hosted compute units were exhausted. The evaluation code remained structurally unchanged except for the model identifier, but it was not run to a completed final measurement. Therefore, no final holdout accuracy belongs to that interrupted attempt.

The completed epoch-loss outputs from that attempt were:

Epoch	Average training loss
1	0.3652072740321358
2	0.2186824376968046
3	0.16869529283201942
4	0.13239717731733497
5	0.10207525314142307
6	0.08553751863058036

Every displayed value is the unweighted mean of per-batch losses for one completed epoch. The downward sequence confirms that the classifier was optimizing on the training batches; it does not by itself establish validation accuracy or generalization.

The hosted notebook interface reported an elapsed cell time of 35 minutes 11 seconds for the six completed epochs before reconnection was required. This timing is environment-specific and must not be generalized to other CPUs, GPUs, batch sizes, or storage pipelines.

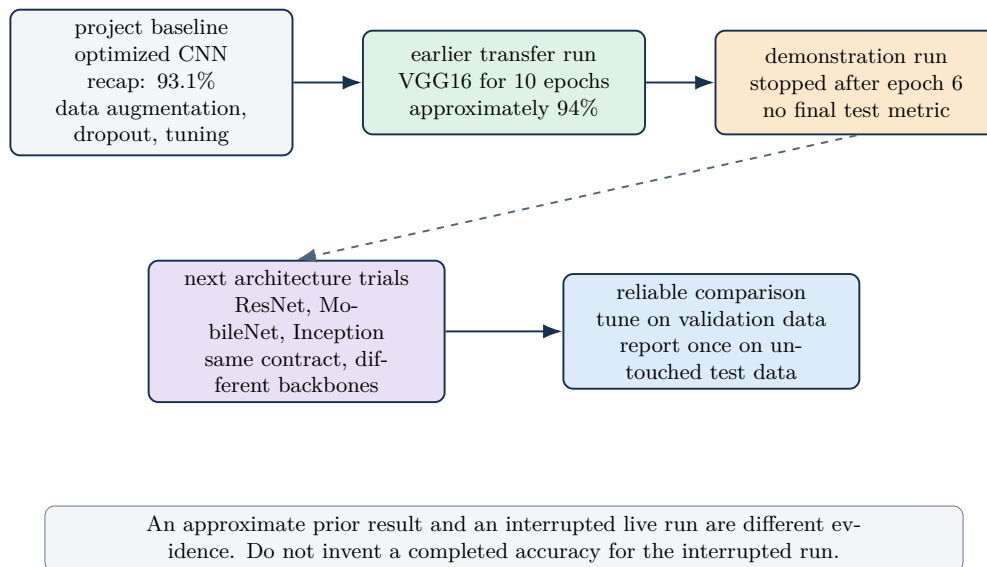


Figure 10: The optimized CNN, approximate earlier transfer result, and interrupted run are distinct evidence; future backbones require a common validation and test protocol. [Technical reference](#).

10.1 A defensible comparison protocol

1. reserve an untouched test partition before experiments;
2. use training data for parameter updates;
3. use validation data for backbone, head, learning-rate, and stopping decisions;

4. keep preprocessing and splits identical across candidate models;
5. report uncertainty across several random seeds when feasible;
6. evaluate the selected configuration once on the untouched test set.

If a 12,000-example test set produced 95% accuracy, the corresponding correct count would be

$$C = 0.95 \times 12,000 = 11,400.$$

This is an interpretation example, not a reported result. It illustrates that a percentage should be connected to an explicit denominator.

10.2 Other pretrained backbones

ResNet adds residual connections that help train deep networks. MobileNet emphasizes efficient depthwise-separable convolutions. Inception mixes receptive-field sizes within a stage. Their internal architectures differ, but the implementation questions remain the same:

- Which weight enum and preprocessing recipe are selected?
- What input channel and spatial shape are required?
- Which module is the source classifier?
- What is the target feature width and class count?
- Which parameters are frozen or unfrozen?
- Which optimizer owns each trainable parameter group?

11 Optional advanced extensions

11.1 Gradual unfreezing

After the new head stabilizes, the last VGG block can be unfrozen. A smaller learning rate should usually be used for pretrained parameters than for the new head. One optimizer can hold parameter groups:

```
for parameter in model.features[24:].parameters():
    parameter.requires_grad = True

optimizer = torch.optim.Adam([
    {
        "params": model.features[24:].parameters(),
        "lr": 1e-5,
    },
    {
        "params": model.classifier.parameters(),
        "lr": 1e-4,
    },
])
```

Listing 9: Fine-tune the last feature block with discriminative learning rates

The first parameter group adapts late source features slowly. The second adapts the target head ten times faster. The index 24 is architecture-specific and must be confirmed by inspecting `model.features`; it must not be copied blindly to another backbone.

11.2 Feature caching

When the backbone is completely frozen and target transforms are deterministic, feature vectors can be computed once and cached. Head training then operates on $[N, 25,088]$ features rather than repeatedly

running VGG16. This can save compute, but dynamic augmentation prevents simple one-time caching because each transformed image may differ each epoch.

11.3 Data augmentation

Random crops, flips, or mild affine transforms can reduce overfitting, but augmentation must preserve class meaning. The deterministic ImageNet resize–center-crop recipe is appropriate for evaluation. Training may use compatible random transforms, while validation and test transforms remain deterministic.

Pitfall

A horizontal flip may be acceptable for many clothing categories, whereas aggressive rotation, color distortion, or cropping can erase discriminative information. Augmentation policy is a modeling decision that belongs to validation, not test-set tuning.

12 Consolidated takeaways and review questions

Takeaway

1. Transfer learning reuses source parameters for a related target task.
2. Fixed-feature extraction freezes the backbone and trains a replacement head; narrow-sense fine-tuning also updates selected pretrained layers.
3. VGG16 requires three-channel ImageNet-compatible input and produces 25,088 flattened features for a 224×224 image.
4. Fashion-MNIST rows must be reshaped, converted to an RGB-compatible representation, resized, cropped, scaled, and normalized in the correct order.
5. PIL consumes HWC arrays; PyTorch models consume CHW samples and BCHW batches.
6. `requires_grad`, optimizer parameter selection, `train()`, `eval()`, and inference mode have separate responsibilities.
7. The source classifier's 1,000 outputs must be replaced by ten target logits.
8. The interrupted training attempt has no final metric; only completed measurements should be reported.
9. Validation data guides model decisions, and an untouched test set supports final reporting.

Review questions

1. What are the source and target tasks in this workflow?
2. Why can early CNN features transfer between related image datasets?
3. What is the difference between fixed-feature extraction and fine-tuning?
4. Why is a ten-class replacement head necessary?
5. Derive $512 \times 7 \times 7 = 25,088$ and explain what each factor means.
6. Why does `np.stack([image] * 3, axis=-1)` produce the correct PIL order?
7. What two operations does `ToTensor` perform for an eight-bit PIL image?
8. Compute the normalized red-channel value for byte 128 using $\mu_R = 0.485$ and $\sigma_R = 0.229$.
9. Why must class-index labels use shape `[B]` and dtype long?

10. Does freezing VGG16 features skip their forward pass? Explain.
11. Why is it clearer to pass only classifier parameters to Adam?
12. Why are `model.eval()` and inference mode both needed?
13. What evidence would be required before claiming that VGG16 reliably exceeds the optimized CNN?
14. When might unfreezing the last feature block be useful?
15. Why must the interrupted six-epoch attempt not be assigned the approximate 94% result from a different completed run?